

I. SUMMARY

The numerical code solves the one-dimensional Schrödinger-Newton equation. This is also the equation that appears at the first non-integrable order of one-dimensional optical turbulence of light propagation through a nematic liquid crystal. The numerical code solves for the 1D complex wave function $\psi = \psi(x, z)$. (Note that z represents the temporal coordinate.)

$$i \frac{\partial \psi}{\partial z} = -\frac{\partial^2 \psi}{\partial x^2} - \frac{1}{2} \psi |\psi|^2 - \frac{1}{2g} \psi \frac{\partial^2 |\psi|^2}{\partial x^2} + F - D,$$

where F and D are additive forcing and dissipation terms defined below. The equation is solved by a pseudo-spectral spatial method in a periodic 1D domain, and time-stepped using an fourth-order exponential time-differencing Runge-Kutta method (see [1, 2] for details).

II. INSTALLATION

The code uses the following libraries that will need to be installed by the user

- gcc (compiler)
- fftw-3 (fast Fourier transform library)
- armadillo (C++ linear algebra library)
- openBLAS (linear algebra library that is used by armadillo)

All of the above can be installed via the local repositories on Linux, or via Macports (and possibly Homebrew) on macOS.

III. EQUATION OF MOTION

The equation solved is given by

$$i \frac{\partial \psi}{\partial z} = -\frac{\partial^2 \psi}{\partial x^2} - \frac{1}{2} \psi |\psi|^2 - \frac{1}{2g} \psi \frac{\partial^2 |\psi|^2}{\partial x^2} + F - D,$$

where the last three terms represent the additive forcing F , and hypo- and hyper-viscosity respectively.

The structure of the additive forcing is such that

$$F = \sum_k f_k \eta_k(t),$$

where $\eta_k(t)$ is a complex Gaussian white noise at mode k and $f_k \in \mathbb{R}$ is the distribution of the forcing amplitudes in Fourier space. The dissipation terms are of the form of hypo- and hyper-viscosities defined through parameters $\alpha, \alpha_{power}, \nu, \nu_{power}$ of the form

$$D = \alpha(-\nabla^2)^{\alpha_{power}} \psi + \nu(-\nabla^2)^{\nu_{power}} \psi,$$

It is assumed that α_{power} takes negative integer values (hypo-viscosity, and linear friction when $\alpha_{power} = 0$), while ν_{power} takes positive integer values (hyper-viscosity).

The system is Hamiltonian in the no forcing and no dissipation limit. It conserve the energy or Hamiltonian given by:

$$H = H_{lin} + H_{non1} + H_{non2} = \int \left[\left| \frac{\partial \psi}{\partial x} \right|^2 - \frac{1}{4} |\psi|^2 + \frac{1}{4g} \left[\frac{\partial |\psi|^2}{\partial x} \right]^2 \right] dx.$$

IV. HOW TO RUN THE CODE

The numerical code contains a `/src/Makefile` that will compile the code using the gcc compiler and create an executable `/src/owt1d`. At the very top are paths to the default locations of the linked libraries for both Linux and MacOS. Simply comment out the one not in use. Bear in mind that the location of the libraries may differ on your local machine.

All global parameters are stored in the header file `/src/const.h`. Any changes in `/src/const.h` will require a recompilation of the numerical code before running.

Once `/src/owt1d` is created, it can be moved to any directory one wishes, but the `/data/` folder must be present in the running directory.

A. The `/data/` folder

To run the code, the user **must** create a `/src/data/` directory in the same folder as the created executable. In the `/data/` folder the user must create a file called `/data/curframe.dat` that will contain two numbers separated by a tab. The numbers represent the current time and the label of the last generated file number. Each time the code outputs a new file for the data, `/data/curframe.dat` will be automatically updated with the new time and label of the last data file. The code uses this information to when restarting from the last generated file.

current time	file number
z	XXXXXX

The folder `./data/` will also contain all the data for the wave function ψ . They are stored in files called `/data/psi.XXXXXX` where XXXXXX is the file number padded by zeros. Files `/data/psi.XXXXXX` are core data for the numerical code.

z	$Re(\psi)$	$Im(\psi)$
$\vdots$	$\vdots$	$\vdots$

B. the `./output/` folder

The code will automatically generate (if not already present) another folder called `./output/` that will contain all necessary in-code post-processing of the data. Usually, this includes the data used for debugging and verification. The code will output files at the same intervals as the main data `/data/psi.XXXXXX`.

`/output/energy.XXXXXX` records the Hamiltonian or energy of the equation to ensure that it is properly conserved by the dynamics. It is recorded in the form of a row of four numbers:

current time	linear energy	nonlinear energy	total (linear + nonlinear) energy
z	H_{lin}	H_{non}	$H_{lin} + H_{non}$

`/output/wave.XXXXXX` records the Hamiltonian or energy of the equation to ensure that it is properly conserved by the dynamics. It is recorded in the form of a row of four numbers:

current time	total wave action	zeroth mode of wave action
z	N	$ \hat{\psi}_{k=0} ^2$

`/output/spec.XXXXXX` records the Fourier wave action spectrum of $|\hat{\psi}_k|^2$ and wave energy spectrum $\omega_k |\hat{\psi}_k|^2 = k^2 |\hat{\psi}_k|^2$ vs k . It is recorded in the form of a five columns of numbers:

$ k $	$ \hat{\psi}_k ^2$	$k^2 \hat{\psi}_k ^2$	$\langle \hat{\psi}_k ^2 \rangle$	$\langle k^2 \hat{\psi}_k ^2 \rangle$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

`/output/flux.XXXXXX` records spectral Fourier flux vs k for both the wave action and energy (instantaneous as well as time averaged over data since start of code). It is recorded in the form of a five columns of numbers:

$ k $	wave action flux η_k	energy flux ϵ_k	$\langle \eta_k \rangle$	$\langle \epsilon_k \rangle$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

/output/dis.XXXXXXX records the total wave action and energy dissipated via the terms that constitute D at the time of output. It is recorded in the form of a row of five numbers:

time	wave action dissipated by α	wave action dissipated by ν	energy dissipated by α	energy dissipated by ν
.	.	.	.	.

Using terminal command `cat energy.* > energy.tot` will append all individual output files into one single file for plotting.

C. the ./initial/ folder

As probably surmised, the code initializes from the data file `/data/psi.XXXXXXX` stored in `/data/` to which the file number in `/data/curframe.dat` points. Therefore, to initialise the code when not restarting one needs to include an initial data file, normally `/src/data/psi.000000` (it requires for `/data/curframe.dat` to be set to 0 0).

If you require a zero-state initial condition because you are running a forced/dissipated simulation, you can set XXXXXX in `/data/curframe.dat` to be any negative integer and the code will interpret this as you wanting to create a zero-state initial condition and will generate this automatically.

If you want to generate a custom (bespoke) initial condition, then next to the `/src/` folder is the `/initial/` folder that contains a simple `.cpp` file for generating a custom initial state that can be copied into `/data/`.

-
- [1] A. Kassam and L. Trefethen, SIAM Journal on Scientific Computing **26**, 1214 (2005).
 - [2] S. M. Cox and P. C. Matthews, Journal of Computational Physics **176**, 430 (2002).